```
const cors = require('cors');
const { Client } = require('pg');
const { v4: uuidv4 } = require('uuid');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const paypal = require('@paypal/checkout-server-sdk');
const nodemailer = require('nodemailer');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const twilio = require('twilio');
const { providerConfig, searchHotels, createHotelReservation } = require('./provider-
adapter');
require('dotenv').config();
const client = new Client({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false }
});

client.connect()
  .then(() => console.log("Connected to PostgreSQL"))
  .catch(err => console.error("PostgreSQL connection error:", err));
const app = express();
app.use(cors());
app.use(express.json());
const providerDocumentsRoutes = require('./routes/providerDocuments');
app.use('/provider/documents', providerDocumentsRoutes);
const JWT_SECRET = process.env.JWT_SECRET || 'secret-key';
function verifyToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.userId = decoded.userId || decoded.id;

    if (!req.userId) {
      return res.status(401).json({ error: 'Invalid token payload' });
    }

    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// Twilio Configuration for SMS
const twilioClient = process.env.TWILIO_ACCOUNT_SID && process.env.TWILIO_AUTH_TOKEN
  ? twilio(process.env.TWILIO_ACCOUNT_SID, process.env.TWILIO_AUTH_TOKEN)
  : null;

// Email Configuration
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS
  }
});

app.post('/auth/signup', async (req, res) => {
  const { name, email, password } = req.body;
```

```
  if (!name || !email || !password) {
    return res.status(400).json({
      error: 'Name, email, and password are required'
    });
  }

  try {
    const normalizedEmail = email.toLowerCase().trim();

    const existing = await client.query(
      'SELECT id FROM users WHERE email = $1',
      [normalizedEmail]
    );

    if (existing.rows.length > 0) {
      return res.status(409).json({
        error: 'Email already registered'
      });
    }

    const id = uuidv4();
    const passwordHash = await bcrypt.hash(password, 12);

    await client.query(
      'INSERT INTO users (id, email, password_hash, name) VALUES ($1, $2, $3, $4)',
      [id, normalizedEmail, passwordHash, name]
    );

    const token = jwt.sign(
      { userId: id, email: normalizedEmail },
      JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.status(201).json({ token, name });
  } catch (err) {
    console.error('Signup error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.post('/auth/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({
      error: 'Email and password are required'
    });
  }

  try {
    const normalizedEmail = email.toLowerCase().trim();

    const result = await client.query(
      'SELECT id, email, name, password_hash FROM users WHERE email = $1',
      [normalizedEmail]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({
        error: 'Invalid email or password'
      });
    }
```

```
    const user = result.rows[0];
    const validPassword = await bcrypt.compare(
      password,
      user.password_hash
    );

    if (!validPassword) {
      return res.status(401).json({
        error: 'Invalid email or password'
      });
    }

    const token = jwt.sign(
      { userId: user.id, email: user.email },
      JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.json({
      token,
      name: user.name
    });
  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/marketplace', async (req, res) => {
  const { country, city, category, type, q } = req.query;
  try {
    const params = []; const filters = ["s.approval_status = 'approved'",
"u.verification_status = 'verified'"];
    if (country) { params.push(country); filters.push(`s.country = $${params.length}`); }
    if (city) { params.push(city); filters.push(`s.city = $${params.length}`); }
    if (category) { params.push(category); filters.push(`s.category = $${params.length}`);
}
    if (type) { params.push(type); filters.push(`s.listing_type = $${params.length}`); }
    if (q) { params.push(`%${q}%`); filters.push(`(s.name ILIKE $${params.length} OR
s.description ILIKE $${params.length})`); }
    const result = await client.query(`SELECT s.id, s.name, s.description, s.category,
s.price, s.country, s.city, s.listing_type, s.rental_unit, s.availability_status, u.id AS
provider_id, u.name AS provider_name, u.rating, u.total_reviews FROM services s JOIN users
u ON u.id=s.provider_id WHERE ${filters.join(' AND ')} ORDER BY s.country, s.city,
s.name`, params);
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.post('/provider/services', verifyToken, async (req, res) => {
  const { name, description, category, duration_minutes = 60, price, country, city,
listing_type = 'service', rental_unit = 'hour' } = req.body;
  if (!name || !description || !category || !price || !country || !city) return
res.status(400).json({ error: 'Required listing fields are missing' });
  if (!['service', 'rental'].includes(listing_type)) return res.status(400).json({ error:
'listing_type must be service or rental' });
  try {
    await client.query('UPDATE users SET is_provider = true, verification_status =
COALESCE(verification_status, $1) WHERE id = $2', ['pending', req.userId]);
    const result = await client.query(`INSERT INTO services (id, provider_id, name,
description, category, duration_minutes, price, country, city, listing_type, rental_unit,
approval_status) VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,'pending') RETURNING *`,
[uuidv4(), req.userId, name, description, category, duration_minutes, price, country,
city, listing_type, rental_unit]);
    res.status(201).json(result.rows[0]);
```

```
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.get('/provider/services', verifyToken, async (req, res) => {
  try { res.json((await client.query('SELECT * FROM services WHERE provider_id=$1 ORDER BY
created_at DESC', [req.userId])).rows); }
  catch (err) { res.status(500).json({ error: err.message }); }
});

app.patch('/admin/provider-listings/:id/approval', verifyAdmin, async (req, res) => {
  const { status } = req.body;
  if (!['approved', 'rejected', 'pending'].includes(status)) return res.status(400).json({
error: 'Invalid approval status' });
  try { res.json((await client.query('UPDATE services SET approval_status=$1 WHERE id=$2
AND provider_id IS NOT NULL RETURNING id,name,approval_status', [status,
req.params.id])).rows[0] || { error: 'Listing not found' }); }
  catch (err) { res.status(500).json({ error: err.message }); }
});
app.post('/provider/register', verifyToken, async (req, res) => {
  try {
    const { bio, phone } = req.body;
    await client.query(
      'UPDATE users SET is_provider = true, bio = $1, phone = $2 WHERE id = $3',
      [bio, phone, req.userId]
    );
    res.json({ message: 'Provider profile created successfully' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/provider/verification-status', verifyToken, async (req, res) => {
  try {
    const result = await client.query('SELECT is_provider, verification_status,
verified_at, verification_notes FROM users WHERE id = $1', [req.userId]);
    res.json(result.rows[0] || { is_provider: false, verification_status: 'unverified' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.get('/admin/providers', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query(`SELECT id, name, email, phone, bio, is_provider,
verification_status, verified_at, verification_notes FROM users WHERE is_provider = true
ORDER BY created_at DESC`);
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.patch('/admin/providers/:id/verification', verifyAdmin, async (req, res) => {
  const { status, notes } = req.body;
  if (!['pending', 'verified', 'rejected', 'unverified'].includes(status)) return
res.status(400).json({ error: 'Invalid verification status' });
  try {
    const result = await client.query(`UPDATE users SET verification_status = $1,
verified_at = CASE WHEN $1 = 'verified' THEN CURRENT_TIMESTAMP ELSE NULL END,
verification_notes = $2 WHERE id = $3 AND is_provider = true RETURNING id, name, email,
verification_status, verified_at, verification_notes`, [status, notes || null,
req.params.id]);
    if (!result.rows.length) return res.status(404).json({ error: 'Provider not found' });
    res.json(result.rows[0]);
  } catch (err) { res.status(500).json({ error: err.message }); }
});


app.get('/provider/:provider_id', async (req, res) => {
```

```
  try {
    const result = await client.query(
      'SELECT id, name, bio, phone, profile_image, rating, total_reviews FROM users WHERE
id = $1 AND is_provider = true',
      [req.params.provider_id]
    );
    if (result.rows.length === 0) return res.status(404).json({ error: 'Provider not
found' });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.patch('/provider/profile', verifyToken, async (req, res) => {
  try {
    const { bio, phone, profile_image } = req.body;
    await client.query(
      'UPDATE users SET bio = $1, phone = $2, profile_image = $3 WHERE id = $4',
      [bio, phone, profile_image, req.userId]
    );
    res.json({ message: 'Profile updated' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Notifications
app.get('/notifications', verifyToken, async (req, res) => {
  try {
    const result = await client.query(
      'SELECT * FROM notifications WHERE user_id = $1 ORDER BY created_at DESC LIMIT 50',
      [req.userId]
    );
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/notifications', verifyToken, async (req, res) => {
  try {
    const { type, title, message } = req.body;
    const notifId = uuidv4();
    await client.query(
      'INSERT INTO notifications (id, user_id, type, title, message) VALUES ($1, $2, $3,
$4, $5)',
      [notifId, req.userId, type, title, message]
    );
    res.json({ message: 'Notification created', notifId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.patch('/notifications/:notif_id/read', verifyToken, async (req, res) => {
  try {
    await client.query(
      'UPDATE notifications SET read = true WHERE id = $1 AND user_id = $2',
      [req.params.notif_id, req.userId]
    );
    res.json({ message: 'Notification marked as read' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
```

```
});

// Booking Calendar
app.post('/booking-events', verifyToken, async (req, res) => {
  try {
    const { booking_id, event_date, event_time, event_type } = req.body;
    const eventId = uuidv4();
    await client.query(
      'INSERT INTO booking_events (id, booking_id, event_date, event_time, event_type)
VALUES ($1, $2, $3, $4, $5)',
      [eventId, booking_id, event_date, event_time, event_type]
    );
    res.json({ message: 'Event created', eventId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/booking-events/:booking_id', verifyToken, async (req, res) => {
  try {
    const result = await client.query(
      'SELECT * FROM booking_events WHERE booking_id = $1 ORDER BY event_date,
event_time',
      [req.params.booking_id]
    );
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Function to send email
async function sendEmail(to, subject, htmlContent)

    await transporter.sendMail({
      from: process.env.EMAIL_USER,
      to,
      subject,
      html: htmlContent
    });
    console.log(`Email sent to ${to}`);
    return true;
  } catch (err) {
    console.error('Email error:', err.message);
    return false;
  }
}


async function sendSMS(phoneNumber, message) {
  if (!twilioClient || !process.env.TWILIO_PHONE_NUMBER)
    console.log('SMS not configured, skipping SMS');
    return false;


  try {
    await twilioClient.messages.create({
      body: message,
      from: process.env.TWILIO_PHONE_NUMBER,
      to: phoneNumber
    });
    console.log(`SMS sent to ${phoneNumber}`);
ication(userId, type, title, message)
  try {
    // Save to DB
```

```
    await client.query(
      'INSERT INTO notifications (id, user_id, type, title, message) VALUES ($1, $2, $3,
$4, $5)',
      [uuidv4(), userId, type, title, message]
    );

    // Get user details for email/SMS
    const userRes = await client.query('SELECT email, phone FROM users WHERE id = $1',
[userId]);
    if (userRes.rows.length === 0) return;

    const user = userRes.rows[0];

    // Send email notification
    if (user.email) {
      await sendEmail(user.email, title, `<p>${message}</p>`);
    }

    // Send SMS notification if phone available
    if (user.phone) {
      await sendSMS(user.phone, `${title}: ${message}`);
    }
  } catch (err) {
    console.error('Notification error:', err.message);
  }
}

let paypalClient;
if (process.env.PAYPAL_CLIENT_ID && process.env.PAYPAL_CLIENT_SECRET) {
  const clientId = process.env.PAYPAL_CLIENT_ID;
  const clientSecret = process.env.PAYPAL_CLIENT_SECRET;

  // Use LiveEnvironment for production, SandboxEnvironment for development
  const environment = process.env.NODE_ENV === 'production'
    ? new paypal.core.LiveEnvironment(clientId, clientSecret)
    : new paypal.core.SandboxEnvironment(clientId, clientSecret);

  paypalClient = new paypal.core.PayPalHttpClient(environment);
  console.log(`? PayPal client initialized (${process.env.NODE_ENV === 'production' ?
'PRODUCTION' : 'SANDBOX'})`);

client.connect()
  .then(() => console.log("Connected to PostgreSQL"))
  .catch(err => console.error("PostgreSQL connection error:", err))

client.connect().catch(err => console.error('Connection error:', err));

async function initDB() {
  try {
    console.log('Starting database initialization...');

    await client.query(`CREATE TABLE IF NOT EXISTS users (id UUID PRIMARY KEY, email
VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, name VARCHAR(255) NOT
NULL, phone VARCHAR(20), bio TEXT, profile_image VARCHAR(500), rating DECIMAL(3,2) DEFAULT
0, total_reviews INT DEFAULT 0, is_provider BOOLEAN DEFAULT false, reset_token
VARCHAR(255), reset_token_expires TIMESTAMP, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP)`);
    console.log('? Users table created with provider fields');
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verification_status
VARCHAR(20) DEFAULT 'unverified'`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verified_at
TIMESTAMP`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verification_notes
TEXT`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verification_status
```

```
VARCHAR(20) DEFAULT 'unverified'`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verified_at
TIMESTAMP`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS verification_notes
TEXT`);

    await client.query(`CREATE TABLE IF NOT EXISTS services (id UUID PRIMARY KEY,
provider_id UUID REFERENCES users(id), name VARCHAR(255) NOT NULL, description TEXT,
category VARCHAR(50) NOT NULL, duration_minutes INT NOT NULL, price DECIMAL(10, 2) NOT
NULL, country VARCHAR(100), city VARCHAR(100), availability_status VARCHAR(50) DEFAULT
'available', available_count INT DEFAULT 999, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP)`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS availability_status
VARCHAR(50) DEFAULT 'available'`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS available_count INT
DEFAULT 999`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS
verification_required BOOLEAN DEFAULT true`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS provider_id UUID`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS
verification_required BOOLEAN DEFAULT true`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS listing_type
VARCHAR(20) DEFAULT 'service'`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS approval_status
VARCHAR(20) DEFAULT 'pending'`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS rental_unit
VARCHAR(50) DEFAULT 'hour'`);
    console.log('? Services table created with provider_id');

    await client.query(`CREATE TABLE IF NOT EXISTS notifications (id UUID PRIMARY KEY,
user_id UUID NOT NULL REFERENCES users(id), type VARCHAR(50), title VARCHAR(255), message
TEXT, read BOOLEAN DEFAULT false, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    console.log('? Notifications table created');


    await client.query(`CREATE TABLE IF NOT EXISTS cities (id UUID PRIMARY KEY, name
VARCHAR(100) NOT NULL UNIQUE, country VARCHAR(100) NOT NULL, region VARCHAR(100), latitude
DECIMAL(10, 8), longitude DECIMAL(11, 8))`);
    console.log('? Cities table created');

    await client.query(`CREATE TABLE IF NOT EXISTS bookings (id UUID PRIMARY KEY, user_id
UUID NOT NULL REFERENCES users(id), service_id UUID NOT NULL REFERENCES services(id),
booking_date DATE NOT NULL, booking_time TIME NOT NULL, address VARCHAR(255), city
VARCHAR(100), latitude DECIMAL(10, 8), longitude DECIMAL(11, 8), user_latitude DECIMAL(10,
8), user_longitude DECIMAL(11, 8), status VARCHAR(50) DEFAULT 'pending',
paypal_transaction_id VARCHAR(255), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS subscriptions (id UUID PRIMARY KEY,
user_id UUID NOT NULL REFERENCES users(id) UNIQUE, plan VARCHAR(50) DEFAULT 'free',
booking_limit INT DEFAULT 10, monthly_bookings INT DEFAULT 0, subscription_date TIMESTAMP
DEFAULT CURRENT_TIMESTAMP, next_reset TIMESTAMP DEFAULT CURRENT_TIMESTAMP + INTERVAL '1
month', renewal_price DECIMAL(10, 2), auto_renew BOOLEAN DEFAULT true,
paypal_subscription_id VARCHAR(255))`);
    await client.query(`CREATE TABLE IF NOT EXISTS payments (id UUID PRIMARY KEY,
booking_id UUID NOT NULL REFERENCES bookings(id), user_id UUID NOT NULL REFERENCES
users(id), service_price DECIMAL(10, 2) NOT NULL, commission DECIMAL(10, 2) NOT NULL,
total_paid DECIMAL(10, 2) NOT NULL, payment_status VARCHAR(50) DEFAULT 'completed',
paypal_transaction_id VARCHAR(255), refund_status VARCHAR(50), refund_amount DECIMAL(10,
2), refunded_at TIMESTAMP, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS earnings (id UUID PRIMARY KEY,
total_commission DECIMAL(12, 2) DEFAULT 0, total_bookings INT DEFAULT 0, last_updated
TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS admin_users (id UUID PRIMARY KEY, email
VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, name VARCHAR(255),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
```

```
    // New tables for enhanced features
    await client.query(`CREATE TABLE IF NOT EXISTS service_images (id UUID PRIMARY KEY,
service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, image_url TEXT NOT
NULL, alt_text VARCHAR(255), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    console.log('? Service images table created');

    await client.query(`CREATE TABLE IF NOT EXISTS service_ratings (id UUID PRIMARY KEY,
service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, user_id UUID NOT NULL
REFERENCES users(id) ON DELETE CASCADE, rating INT NOT NULL CHECK (rating >= 1 AND rating
<= 5), comment TEXT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    console.log('? Service ratings table created');

    await client.query(`CREATE TABLE IF NOT EXISTS provider_availability (id UUID PRIMARY
KEY, provider_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, date DATE NOT NULL,
start_time TIME NOT NULL, end_time TIME NOT NULL, is_available BOOLEAN DEFAULT true,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`ALTER TABLE bookings ADD COLUMN IF NOT EXISTS timezone
VARCHAR(100) DEFAULT 'UTC'`);
    await client.query(`ALTER TABLE bookings ADD COLUMN IF NOT EXISTS buffer_minutes INT
DEFAULT 0`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS buffer_minutes INT
DEFAULT 15`);
    await client.query(`CREATE TABLE IF NOT EXISTS availability_slots (id UUID PRIMARY
KEY, service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, provider_id UUID
REFERENCES users(id) ON DELETE SET NULL, slot_date DATE NOT NULL, start_time TIME NOT
NULL, end_time TIME NOT NULL, timezone VARCHAR(100) DEFAULT 'UTC', capacity INT DEFAULT 1,
active BOOLEAN DEFAULT true, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS recurring_schedules (id UUID PRIMARY
KEY, service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, provider_id UUID
REFERENCES users(id) ON DELETE SET NULL, weekday INT NOT NULL CHECK (weekday BETWEEN 0 AND
6), start_time TIME NOT NULL, end_time TIME NOT NULL, slot_minutes INT DEFAULT 60,
buffer_minutes INT DEFAULT 15, timezone VARCHAR(100) DEFAULT 'UTC', valid_from DATE NOT
NULL, valid_until DATE, capacity INT DEFAULT 1, active BOOLEAN DEFAULT true)`);
    await client.query(`CREATE TABLE IF NOT EXISTS schedule_breaks (id UUID PRIMARY KEY,
schedule_id UUID NOT NULL REFERENCES recurring_schedules(id) ON DELETE CASCADE,
break_start TIME NOT NULL, break_end TIME NOT NULL)`);
    await client.query(`CREATE TABLE IF NOT EXISTS schedule_exceptions (id UUID PRIMARY
KEY, service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, exception_date
DATE NOT NULL, closed BOOLEAN DEFAULT true, start_time TIME, end_time TIME)`);
    await client.query(`CREATE INDEX IF NOT EXISTS recurring_schedule_lookup ON
recurring_schedules(service_id, weekday, valid_from, active)`);
    await client.query(`CREATE INDEX IF NOT EXISTS availability_slots_lookup ON
availability_slots(service_id, slot_date, start_time)`);
    console.log('? Provider availability table created');

    await client.query(`CREATE TABLE IF NOT EXISTS booking_confirmations (id UUID PRIMARY
KEY, booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE, user_id UUID NOT
NULL REFERENCES users(id) ON DELETE CASCADE, email_sent BOOLEAN DEFAULT false, sent_at
TIMESTAMP, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS provider_documents (id UUID PRIMARY
KEY, provider_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, document_type
VARCHAR(50) NOT NULL, document_url TEXT NOT NULL, document_name VARCHAR(255), status
VARCHAR(20) DEFAULT 'pending', review_notes TEXT, reviewed_at TIMESTAMP, created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    await client.query(`CREATE TABLE IF NOT EXISTS booking_reminders (id UUID PRIMARY KEY,
booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE, reminder_type
VARCHAR(30) NOT NULL, scheduled_for TIMESTAMP NOT NULL, sent_at TIMESTAMP,
UNIQUE(booking_id, reminder_type))`);
    await client.query(`CREATE INDEX IF NOT EXISTS booking_reminders_due ON
booking_reminders(scheduled_for, sent_at)`);
    console.log('? Booking confirmations table created');

        await client.query(`CREATE TABLE IF NOT EXISTS reviews (id UUID PRIMARY KEY,
service_id UUID NOT NULL REFERENCES services(id) ON DELETE CASCADE, booking_id UUID
REFERENCES bookings(id) ON DELETE SET NULL, user_id UUID NOT NULL REFERENCES users(id) ON
```

```
DELETE CASCADE, rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5), review_text TEXT
NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)`);
    console.log('Reviews table created');

    // Initialize earnings tracker
    await client.query(`INSERT INTO earnings (id) SELECT '00000000-0000-0000-0000-
000000000000' WHERE NOT EXISTS (SELECT 1 FROM earnings)`);
    console.log('? Subscriptions, payments, and earnings tables created');
    console.log('? Admin users table created');
    console.log('? Bookings table created');

    console.log('Database initialization complete!');
  } catch (err) {
    console.error('DB init error:', err.message);
  }
}

app.post('/reviews', verifyToken, async (req, res) => {
  const { service_id, booking_id, rating, review_text } = req.body;
  try {
    if (rating < 1 || rating > 5) return res.status(400).json({ error: 'Rating must be 1-
5' });

    const reviewId = uuidv4();
    await client.query(
      'INSERT INTO reviews (id, service_id, booking_id, user_id, rating, review_text)
VALUES ($1, $2, $3, $4, $5, $6)',
      [reviewId, service_id, booking_id, req.userId, rating, review_text]
    );
    res.json({ message: 'Review added successfully', reviewId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/reviews/:service_id', async (req, res) => {
  const { service_id } = req.params;
  try {
    const result = await client.query(
      'SELECT r.*, u.name FROM reviews r JOIN users u ON r.user_id = u.id WHERE
r.service_id = $1 ORDER BY r.created_at DESC',
      [service_id]
    );

    // Calculate average rating
    const avgResult = await client.query(
      'SELECT AVG(rating) as avg_rating, COUNT(*) as total_reviews FROM reviews WHERE
service_id = $1',
      [service_id]
    );

    res.json({
      reviews: result.rows,
      average_rating: parseFloat(avgResult.rows[0].avg_rating) || 0,
      total_reviews: parseInt(avgResult.rows[0].total_reviews) || 0
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Service Availability
app.patch('/services/:service_id/availability', verifyToken, async (req, res) => {
  const { service_id } = req.params;
  const { availability_status, available_count } = req.body;
```

```
  try {
    await client.query(
      'UPDATE services SET availability_status = $1, available_count = $2 WHERE id = $3',
      [availability_status, available_count, service_id]
    );
    res.json({ message: 'Availability updated' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/services/:service_id/availability', async (req, res) => {
  const { service_id } = req.params;
  try {
    const result = await client.query(
      'SELECT id, name, availability_status, available_count FROM services WHERE id = $1',
      [service_id]
    );
    res.json(result.rows[0] || { error: 'Service not found' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Service Images
app.post('/services/:service_id/images', verifyToken, async (req, res) => {
  const { service_id } = req.params;
  const { image_url, description } = req.body;
  try {
    const imageId = uuidv4();
    await client.query(
      'INSERT INTO service_images (id, service_id, image_url, description) VALUES ($1, $2,
$3, $4)',
      [imageId, service_id, image_url, description]
    );
    res.json({ message: 'Image added', imageId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/services/:service_id/images', async (req, res) => {
  const { service_id } = req.params;
  try {
    const result = await client.query(
      'SELECT id, image_url, description FROM service_images WHERE service_id = $1 ORDER
BY created_at DESC',
      [service_id]
    );
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Provider Availability Calendar
app.post('/provider-availability', verifyToken, async (req, res) => {
  const { available_date, start_time, end_time } = req.body;
  try {
    const availId = uuidv4();
    await client.query(
      'INSERT INTO provider_availability (id, user_id, available_date, start_time,
end_time) VALUES ($1, $2, $3, $4, $5)',
      [availId, req.userId, available_date, start_time, end_time]
    );
```

```
    res.json({ message: 'Availability added', availId });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/provider-availability/:user_id', async (req, res) => {
  const { user_id } = req.params;
  try {
    const result = await client.query(
      'SELECT available_date, start_time, end_time FROM provider_availability WHERE
user_id = $1 AND available_date >= CURRENT_DATE ORDER BY available_date, start_time',
      [user_id]
    );
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete('/provider-availability/:avail_id', verifyToken, async (req, res) => {
  const { avail_id } = req.params;
  try {
    await client.query(
      'DELETE FROM provider_availability WHERE id = $1 AND user_id = $2',
      [avail_id, req.userId]
    );
    res.json({ message: 'Availability deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/services/:service_id/quote', async (req, res) => {
  try {
    const result = await client.query(
      `SELECT id, name, price, availability_status, available_count
       FROM services WHERE id = $1`,
      [req.params.service_id]
    );
    if (!result.rows.length) return res.status(404).json({ error: 'Service not found' });
    const service = result.rows[0];
    const servicePrice = Number(service.price);
    const commission = Number((servicePrice * 0.20).toFixed(2));
    res.json({ ...service, service_price: servicePrice, commission, total:
Number((servicePrice + commission).toFixed(2)), available: service.availability_status ===
'available' && service.available_count > 0 });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/bookings/:id/confirmation', verifyToken, async (req, res) => {
  try {
    const result = await client.query(
      `SELECT b.id, b.booking_date, b.booking_time, b.address, b.city, b.status,
              s.name AS service_name, s.price, u.name AS customer_name,
              bc.email_sent, bc.sent_at
       FROM bookings b JOIN services s ON s.id = b.service_id JOIN users u ON u.id =
b.user_id
       LEFT JOIN booking_confirmations bc ON bc.booking_id = b.id
       WHERE b.id = $1 AND b.user_id = $2`,
      [req.params.id, req.userId]
    );
    if (!result.rows.length) return res.status(404).json({ error: 'Booking not found' });
```

```
    const booking = result.rows[0];
    const servicePrice = Number(booking.price);
    res.json({ ...booking, service_price: servicePrice, commission: Number((servicePrice *
0.20).toFixed(2)), total: Number((servicePrice * 1.20).toFixed(2)) });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/bookings', verifyToken, async (req, res) => {
  const { service_id, booking_date, booking_time, address, city, latitude, longitude } =
req.body;
  if (!service_id || !booking_date || !booking_time || !address || !city) {
    return res.status(400).json({ error: 'Service, date, time, address, and city are
required' });
  }
  try {
    await client.query('BEGIN');
    const serviceResult = await client.query(
      `SELECT id, name, price, city FROM services
       WHERE id = $1 AND availability_status = 'available' AND available_count > 0
       FOR UPDATE`,
      [service_id]
    );
    if (!serviceResult.rows.length) {
      await client.query('ROLLBACK');
      return res.status(409).json({ error: 'Service is currently unavailable' });
    }
    const slot = await client.query(`SELECT s.id, s.capacity FROM availability_slots s
WHERE s.service_id=$1 AND s.slot_date=$2 AND s.start_time <= $3::time AND s.end_time >
$3::time AND s.active=true FOR UPDATE`, [service_id, booking_date, booking_time]);
    if (slot.rows.length) {
      const used = await client.query(`SELECT COUNT(*)::int AS count FROM bookings WHERE
service_id=$1 AND booking_date=$2 AND booking_time >= $3 AND status IN
('pending','confirmed')`, [service_id, booking_date, booking_time]);
      if (used.rows[0].count >= slot.rows[0].capacity) { await client.query('ROLLBACK');
return res.status(409).json({ error: 'Selected slot is full' }); }
    }

    const conflict = await client.query(
      `SELECT b.id FROM bookings b JOIN services s ON s.id = b.service_id
       WHERE b.service_id = $1 AND b.booking_date = $2 AND b.status IN
('pending','confirmed')
       AND b.booking_time < ($4::time + (s.duration_minutes + s.buffer_minutes) * interval
'1 minute')
       AND (b.booking_time + (s.duration_minutes + s.buffer_minutes) * interval '1
minute') > $3::time
       FOR UPDATE`,
      [service_id, booking_date, booking_time, booking_time]
    );
    if (conflict.rows.length) {
      await client.query('ROLLBACK');
      return res.status(409).json({ error: 'This time conflicts with an existing
reservation' });
    }
    const service = serviceResult.rows[0];
    const bookingId = uuidv4();
    await client.query(
      `INSERT INTO bookings (id, user_id, service_id, booking_date, booking_time, address,
city, latitude, longitude)
       VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)`,
      [bookingId, req.userId, service_id, booking_date, booking_time, address, city,
latitude || null, longitude || null]
    );
    await client.query('UPDATE services SET available_count = available_count - 1 WHERE id
```

```
= $1', [service_id]);
    await client.query(
      `INSERT INTO booking_confirmations (id, booking_id, user_id, email_sent, sent_at)
VALUES ($1, $2, $3, false, NULL)`,
      [uuidv4(), bookingId, req.userId]
    );
    await client.query(`INSERT INTO booking_reminders (id, booking_id, reminder_type,
scheduled_for) VALUES ($1,$2,'24h', NOW() + interval '24 hours') ON CONFLICT DO NOTHING`,
[uuidv4(), bookingId]);
    await client.query(`INSERT INTO booking_reminders (id, booking_id, reminder_type,
scheduled_for) VALUES ($1,$2,'1h', NOW() + interval '1 hour') ON CONFLICT DO NOTHING`,
[uuidv4(), bookingId]);
    await client.query('COMMIT');
    const servicePrice = Number(service.price);
    res.status(201).json({ id: bookingId, service_name: service.name, service_price:
servicePrice, commission: Number((servicePrice * 0.20).toFixed(2)), total:
Number((servicePrice * 1.20).toFixed(2)), status: 'pending', confirmation:
`/bookings/${bookingId}/confirmation` });
  } catch (err) {
    await client.query('ROLLBACK').catch(() => null);
    console.error('Booking creation error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.delete('/bookings/:id', verifyToken, async (req, res) => {
  const { id } = req.params;
  try {
    await client.query('UPDATE bookings SET status = $1 WHERE id = $2 AND user_id = $3',
['cancelled', id, req.userId]);
    res.json({ message: 'Booking cancelled' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// PayPal Payment Endpoints
app.post('/payments/paypal/create-order', verifyToken, async (req, res) => {
  const { booking_id, amount } = req.body;

  if (!process.env.PAYPAL_CLIENT_ID || !process.env.PAYPAL_CLIENT_SECRET) {
    return res.status(500).json({ error: 'PayPal credentials not configured' });
  }

  try {
    const auth = Buffer.from(process.env.PAYPAL_CLIENT_ID + ':' +
process.env.PAYPAL_CLIENT_SECRET).toString('base64');
    const baseURL = process.env.NODE_ENV === 'production'
      ? 'https://api.paypal.com/v2/checkout/orders'
      : 'https://api.sandbox.paypal.com/v2/checkout/orders';

    const response = await fetch(baseURL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Basic ${auth}`
      },
      body: JSON.stringify({
        intent: 'CAPTURE',
        purchase_units: [
          {
            amount: {
              currency_code: 'USD',
              value: amount.toFixed(2)
            },
```

```
            description: `Booking ID: ${booking_id}`
          }
        ]
      })
    });

    const data = await response.json();

    console.log('PayPal Order Created:', { status: response.status, orderId: data.id,
error: data.message });

    if (response.ok && data.id) {
      res.json({ id: data.id });
    } else {
      console.error('PayPal Error:', data);
      res.status(400).json({ error: 'Failed to create PayPal order', details: data });
    }
  } catch (err) {
    console.error('PayPal Create Order Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.post('/payments/paypal/capture-order', verifyToken, async (req, res) => {
  const { order_id, booking_id } = req.body;

  if (!process.env.PAYPAL_CLIENT_ID || !process.env.PAYPAL_CLIENT_SECRET) {
    return res.status(500).json({ error: 'PayPal credentials not configured' });
  }

  try {
    const auth = Buffer.from(process.env.PAYPAL_CLIENT_ID + ':' +
process.env.PAYPAL_CLIENT_SECRET).toString('base64');
    const baseURL = process.env.NODE_ENV === 'production'
      ? `https://api.paypal.com/v2/checkout/orders/${order_id}/capture`
      : `https://api.sandbox.paypal.com/v2/checkout/orders/${order_id}/capture`;

    const response = await fetch(baseURL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Basic ${auth}`
      }
    });

    const data = await response.json();

    console.log('PayPal Order Captured:', { status: response.status, orderId: order_id });

    if (response.ok && data.status === 'COMPLETED') {
      // Get transaction ID from response
      const transactionId = data.purchase_units?.[0]?.payments?.captures?.[0]?.id ||
order_id;

      // Update payment status in database
      await client.query(
        'UPDATE payments SET payment_status = $1, paypal_transaction_id = $2 WHERE
booking_id = $3',
        ['completed', transactionId, booking_id]
      );

      // Store transaction ID in booking for refunds
      await client.query(
        'UPDATE bookings SET paypal_transaction_id = $1 WHERE id = $2',
        [transactionId, booking_id]
```

```
      );

      // Get booking and user details for email
      const bookingRes = await client.query(
        'SELECT b.*, s.name as service_name, s.price, u.email, u.name FROM bookings b JOIN
services s ON b.service_id = s.id JOIN users u ON b.user_id = u.id WHERE b.id = $1',
        [booking_id]
      );

      if (bookingRes.rows.length > 0) {
        const booking = bookingRes.rows[0];
        const commission = booking.price * 0.20;

        // Send email receipt
        const emailHTML = `
          <h2>?? Payment Receipt - BookHub</h2>
          <p>Hello <strong>${booking.name}</strong>,</p>
          <p>Your payment has been successfully processed!</p>
          <hr>
          <h3>Booking Details:</h3>
          <ul>
            <li><strong>Service:</strong> ${booking.service_name}</li>
            <li><strong>Date:</strong> ${booking.booking_date}</li>
            <li><strong>Time:</strong> ${booking.booking_time}</li>
            <li><strong>Location:</strong> ${booking.address}, ${booking.city}</li>
            <li><strong>Service Price:</strong> $${parseFloat(booking.price).toFixed(2)}
</li>
            <li><strong>Your Commission (20%):</strong> $${commission.toFixed(2)}</li>
            <li><strong>Total Paid:</strong> $${(parseFloat(booking.price) +
commission).toFixed(2)}</li>
          </ul>
          <hr>
          <p><strong>Status:</strong> ? Payment Completed</p>
          <p>Order ID: ${booking_id}</p>
          <p>PayPal Transaction ID: ${order_id}</p>
          <hr>
          <p>Thank you for booking with BookHub!</p>
          <p>Questions? Contact us at support@bookhub.app</p>
        `;

        await sendEmail(booking.email, 'Payment Receipt - BookHub Booking', emailHTML);

        // Send booking confirmation notification
        await sendNotification(
          req.userId,
          'booking_confirmed',
          '?? Booking Confirmed!',
          `Your booking for ${booking.service_name} on ${booking.booking_date} has been
confirmed. Payment successful!`
        );
      }

      res.json({ status: 'COMPLETED', message: 'Payment captured successfully' });
    } else {
      console.error('PayPal Capture Error:', data);
      res.status(400).json({ error: 'Payment capture failed', status: data.status });
    }
  } catch (err) {
    console.error('PayPal Capture Order Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/payments/paypal/config', (req, res) => {
  res.json({
```

```
    clientId: process.env.PAYPAL_CLIENT_ID || 'test_client_id'
  });
});

// STRIPE PAYMENT ENDPOINTS
app.post('/payments/stripe/create-payment-intent', verifyToken, async (req, res) => {
  const { bookingId, amount } = req.body;

  try {
    const paymentIntent = await stripe.paymentIntents.create({
      amount: Math.round(parseFloat(amount) * 100), // Convert to cents
      currency: 'usd',
      metadata: {
        bookingId,
        userId: req.userId
      },
      description: `BookHub Booking - ${bookingId}`
    });

    res.json({ clientSecret: paymentIntent.client_secret, paymentIntentId:
paymentIntent.id });
  } catch (err) {
    console.error('Stripe Payment Intent Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.post('/payments/stripe/confirm-payment', verifyToken, async (req, res) => {
  const { paymentIntentId, bookingId } = req.body;

  try {
    const paymentIntent = await stripe.paymentIntents.retrieve(paymentIntentId);

    if (paymentIntent.status !== 'succeeded') {
      return res.status(400).json({ error: 'Payment not successful', status:
paymentIntent.status });
    }

    // Get booking and service details
    const bookingRes = await client.query(
      'SELECT b.*, s.price, s.name as service_name, u.email, u.name FROM bookings b JOIN
services s ON b.service_id = s.id JOIN users u ON b.user_id = u.id WHERE b.id = $1 AND
b.user_id = $2',
      [bookingId, req.userId]
    );

    if (bookingRes.rows.length === 0) return res.status(404).json({ error: 'Booking not
found' });

    const booking = bookingRes.rows[0];
    const servicePrice = parseFloat(booking.price);
    const commission = servicePrice * 0.20;
    const totalPaid = servicePrice + commission;

    // Create payment record
    const paymentId = uuidv4();
    await client.query(
      'INSERT INTO payments (id, booking_id, user_id, service_price, commission,
total_paid, payment_status, paypal_transaction_id) VALUES ($1, $2, $3, $4, $5, $6, $7,
$8)',
      [paymentId, bookingId, req.userId, servicePrice, commission, totalPaid, 'completed',
paymentIntent.charges.data[0].id]
    );

    // Update booking status
```

```
    await client.query('UPDATE bookings SET status = $1 WHERE id = $2', ['confirmed',
bookingId]);

    // Update earnings
    await client.query(
      'UPDATE earnings SET total_commission = total_commission + $1, total_bookings =
total_bookings + 1 WHERE id = $2',
      [commission, '00000000-0000-0000-0000-000000000000']
    );

    // Send email receipt
    const emailHTML = `
      <h2>?? Payment Receipt - BookHub</h2>
      <p>Hello <strong>${booking.name}</strong>,</p>
      <p>Your payment has been successfully processed!</p>
      <hr>
      <h3>Booking Details:</h3>
      <ul>
        <li><strong>Service:</strong> ${booking.service_name}</li>
        <li><strong>Date:</strong> ${booking.booking_date}</li>
        <li><strong>Time:</strong> ${booking.booking_time}</li>
        <li><strong>Location:</strong> ${booking.address}, ${booking.city}</li>
        <li><strong>Service Price:</strong> $${servicePrice.toFixed(2)}</li>
        <li><strong>Your Commission (20%):</strong> $${commission.toFixed(2)}</li>
        <li><strong>Total Paid:</strong> $${totalPaid.toFixed(2)}</li>
      </ul>
      <hr>
      <p><strong>Status:</strong> ? Payment Completed</p>
      <p>Payment Method: ?? Stripe Card Payment</p>
      <p>Transaction ID: ${paymentIntent.charges.data[0].id}</p>
      <hr>
      <p>Thank you for booking with BookHub!</p>
      <p>Questions? Contact us at support@bookhub.app</p>
    `;

    await sendEmail(booking.email, 'Payment Receipt - BookHub Booking', emailHTML);

    res.json({ message: 'Payment confirmed successfully', paymentId, status: 'completed'
});
  } catch (err) {
    console.error('Stripe Confirm Payment Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/payments/stripe/config', (req, res) => {
  res.json({
    publishableKey: process.env.STRIPE_PUBLISHABLE_KEY
  });
});

// REFUND ENDPOINTS
app.post('/payments/:paymentId/refund', verifyToken, async (req, res) => {
  const { paymentId } = req.params;

  try {
    // Get payment details
    const paymentRes = await client.query('SELECT * FROM payments WHERE id = $1 AND
user_id = $2', [paymentId, req.userId]);
    if (paymentRes.rows.length === 0) return res.status(404).json({ error: 'Payment not
found' });

    const payment = paymentRes.rows[0];
    if (payment.payment_status === 'refunded') return res.status(400).json({ error:
'Payment already refunded' });
```

```
    // Get booking and booking's user
    const bookingRes = await client.query('SELECT id, user_id FROM bookings WHERE id =
$1', [payment.booking_id]);
    const booking = bookingRes.rows[0];

    // Only allow refund by payment owner or admin
    if (req.userId !== payment.user_id) {
      return res.status(403).json({ error: 'Unauthorized' });
    }

    // Process PayPal refund (using stored transaction ID from payment)
    const auth = Buffer.from(process.env.PAYPAL_CLIENT_ID + ':' +
process.env.PAYPAL_CLIENT_SECRET).toString('base64');

    // Get the transaction ID from the booking
    const txnRes = await client.query('SELECT paypal_transaction_id FROM bookings WHERE id
= $1', [payment.booking_id]);
    const txnId = txnRes.rows[0]?.paypal_transaction_id;

    if (!txnId) {
      return res.status(400).json({ error: 'Cannot refund this payment - no transaction ID
found' });
    }

    // Call PayPal refund API
    const refundRes = await
fetch(`https://api.sandbox.paypal.com/v2/payments/captures/${txnId}/refund`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Basic ${auth}`
      },
      body: JSON.stringify({
        amount: {
          currency_code: 'USD',
          value: payment.total_paid.toString()
        },
        note_to_payer: 'Booking refund'
      })
    });

    const refundData = await refundRes.json();

    if (!refundRes.ok) {
      console.error('PayPal Refund Error:', refundData);
      return res.status(400).json({ error: 'Refund failed', details: refundData });
    }

    // Update payment status to refunded
    await client.query('UPDATE payments SET payment_status = $1 WHERE id = $2',
['refunded', paymentId]);

    // Update booking status to cancelled
    await client.query('UPDATE bookings SET status = $1 WHERE id = $2', ['cancelled',
payment.booking_id]);

    // Deduct from earnings
    await client.query(
      'UPDATE earnings SET total_commission = total_commission - $1 WHERE id = $2',
      [payment.commission, '00000000-0000-0000-0000-000000000000']
    );

    // Send refund email to user
    const userRes = await client.query('SELECT email FROM users WHERE id = $1',
```

```
[req.userId]);
    const userEmail = userRes.rows[0]?.email;

    const emailHtml = `
      <h2>Refund Processed!</h2>
      <p>Your refund of <strong>$${payment.total_paid}</strong> has been processed
successfully.</p>
      <p>Refund ID: ${refundData.id}</p>
      <p>The funds should appear in your PayPal account within 3-5 business days.</p>
      <p>Booking Status: Cancelled</p>
      <p>Thank you for using BookHub!</p>
    `;
    await sendEmail(userEmail, 'BookHub: Refund Processed', emailHtml);

    res.json({ message: 'Refund processed successfully', refundId: refundData.id });
  } catch (err) {
    console.error('Refund Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/payments/:paymentId/refund-status', verifyToken, async (req, res) => {
  const { paymentId } = req.params;

  try {
    const result = await client.query('SELECT payment_status FROM payments WHERE id = $1
AND user_id = $2', [paymentId, req.userId]);
    if (result.rows.length === 0) return res.status(404).json({ error: 'Payment not found'
});

    res.json({ status: result.rows[0].payment_status });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// REFUND REQUEST ENDPOINTS (Admin Approval Required)
app.post('/payments/:paymentId/request-refund', verifyToken, async (req, res) => {
  const { paymentId } = req.params;
  const { refundAmount, reason } = req.body;

  try {
    // Get payment details
    const paymentRes = await client.query('SELECT * FROM payments WHERE id = $1 AND
user_id = $2', [paymentId, req.userId]);
    if (paymentRes.rows.length === 0) return res.status(404).json({ error: 'Payment not
found' });

    const payment = paymentRes.rows[0];
    if (payment.payment_status !== 'completed') return res.status(400).json({ error: 'Only
completed payments can be refunded' });
    if (payment.refund_status === 'pending' || payment.refund_status === 'approved')
return res.status(400).json({ error: 'Refund already requested' });

    // Check 7-day refund window
    const createdDate = new Date(payment.created_at);
    const now = new Date();
    const daysDiff = Math.floor((now - createdDate) / (1000 * 60 * 60 * 24));

    if (daysDiff > 7) {
      return res.status(400).json({ error: `Refund window expired. You can only refund
within 7 days of payment. Payment was made ${daysDiff} days ago.` });
    }

    // Validate refund amount
```

```
    const refundAmt = parseFloat(refundAmount) || parseFloat(payment.total_paid);
    if (refundAmt > parseFloat(payment.total_paid)) {
      return res.status(400).json({ error: 'Refund amount cannot exceed payment amount'
});
    }
    if (refundAmt <= 0) {
      return res.status(400).json({ error: 'Refund amount must be greater than 0' });
    }

    // Create refund request (pending admin approval)
    await client.query(
      'UPDATE payments SET refund_status = $1, refund_amount = $2 WHERE id = $3',
      ['pending', refundAmt, paymentId]
    );

    // Send email to user
    const userRes = await client.query('SELECT email FROM users WHERE id = $1',
[req.userId]);
    const userEmail = userRes.rows[0]?.email;

    const emailHtml = `
      <h2>Refund Request Submitted</h2>
      <p>Your refund request for <strong>$${refundAmt.toFixed(2)}</strong> has been
submitted.</p>
      <p><strong>Reason:</strong> ${reason || 'No reason provided'}</p>
      <p><strong>Status:</strong> Pending Admin Approval</p>
      <p>Our admin team will review your request and process it within 24-48 hours.</p>
      <p>Thank you for your patience!</p>
    `;
    await sendEmail(userEmail, 'BookHub: Refund Request Received', emailHtml);

    res.json({ message: 'Refund request submitted for admin approval', status: 'pending'
});
  } catch (err) {
    console.error('Refund Request Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/admin/refund-requests', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query(`
      SELECT p.id, p.booking_id, p.user_id, p.total_paid, p.refund_amount,
p.refund_status, p.created_at,
        b.city, s.name as service_name,
        u.name as user_name, u.email
      FROM payments p
      JOIN bookings b ON p.booking_id = b.id
      JOIN services s ON b.service_id = s.id
      JOIN users u ON p.user_id = u.id
      WHERE p.refund_status IN ('pending', 'approved', 'rejected')
      ORDER BY p.created_at DESC
    `);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.patch('/admin/refund-requests/:paymentId/approve', verifyAdmin, async (req, res) => {
  const { paymentId } = req.params;

  try {
    // Get payment details
    const paymentRes = await client.query('SELECT * FROM payments WHERE id = $1',
```

```
[paymentId]);
    if (paymentRes.rows.length === 0) return res.status(404).json({ error: 'Payment not
found' });

    const payment = paymentRes.rows[0];
    if (payment.refund_status !== 'pending') return res.status(400).json({ error: 'Only
pending refunds can be approved' });

    const refundAmount = payment.refund_amount;
    const txnRes = await client.query('SELECT paypal_transaction_id FROM bookings WHERE id
= $1', [payment.booking_id]);
    const txnId = txnRes.rows[0]?.paypal_transaction_id;

    if (!txnId) return res.status(400).json({ error: 'No transaction ID found' });

    // Process PayPal refund
    const auth = Buffer.from(process.env.PAYPAL_CLIENT_ID + ':' +
process.env.PAYPAL_CLIENT_SECRET).toString('base64');

    const refundRes = await
fetch(`https://api.sandbox.paypal.com/v2/payments/captures/${txnId}/refund`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Basic ${auth}`
      },
      body: JSON.stringify({
        amount: {
          currency_code: 'USD',
          value: refundAmount.toString()
        },
        note_to_payer: 'Approved refund request'
      })
    });

    const refundData = await refundRes.json();
    if (!refundRes.ok) return res.status(400).json({ error: 'PayPal refund failed',
details: refundData });

    // Update payment
    await client.query(
      'UPDATE payments SET refund_status = $1, refunded_at = NOW() WHERE id = $2',
      ['approved', paymentId]
    );

    // Deduct from earnings if full refund
    if (parseFloat(refundAmount) === parseFloat(payment.total_paid)) {
      await client.query(
        'UPDATE earnings SET total_commission = total_commission - $1 WHERE id = $2',
        [payment.commission, '00000000-0000-0000-0000-000000000000']
      );
      await client.query('UPDATE bookings SET status = $1 WHERE id = $2', ['cancelled',
payment.booking_id]);
    }

    // Send approval email
    const userRes = await client.query('SELECT email FROM users WHERE id = $1',
[payment.user_id]);
    const userEmail = userRes.rows[0]?.email;

    const emailHtml = `
      <h2>Refund Approved!</h2>
      <p>Your refund of <strong>$${refundAmount.toFixed(2)}</strong> has been approved and
processed.</p>
      <p>Refund ID: ${refundData.id}</p>
```

```
      <p>The funds should appear in your PayPal account within 3-5 business days.</p>
      <p>Thank you for using BookHub!</p>
    `;
    await sendEmail(userEmail, 'BookHub: Refund Approved', emailHtml);

    res.json({ message: 'Refund approved and processed', refundId: refundData.id });
  } catch (err) {
    console.error('Admin Refund Approval Error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.patch('/admin/refund-requests/:paymentId/reject', verifyAdmin, async (req, res) => {
  const { paymentId } = req.params;
  const { reason } = req.body;

  try {
    const paymentRes = await client.query('SELECT * FROM payments WHERE id = $1',
[paymentId]);
    if (paymentRes.rows.length === 0) return res.status(404).json({ error: 'Payment not
found' });

    const payment = paymentRes.rows[0];
    if (payment.refund_status !== 'pending') return res.status(400).json({ error: 'Only
pending refunds can be rejected' });

    await client.query('UPDATE payments SET refund_status = $1 WHERE id = $2',
['rejected', paymentId]);

    // Send rejection email
    const userRes = await client.query('SELECT email FROM users WHERE id = $1',
[payment.user_id]);
    const userEmail = userRes.rows[0]?.email;

    const emailHtml = `
      <h2>Refund Request Rejected</h2>
      <p>Your refund request of <strong>$${payment.refund_amount.toFixed(2)}</strong> has
been rejected.</p>
      <p><strong>Reason:</strong> ${reason || 'No reason provided'}</p>
      <p>If you have any questions, please contact our support team.</p>
    `;
    await sendEmail(userEmail, 'BookHub: Refund Request Rejected', emailHtml);

    res.json({ message: 'Refund request rejected' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ADMIN PANEL ENDPOINTS
const ADMIN_EMAIL = 'admin@bookhub.app';
const ADMIN_PASSWORD = 'Admin@BookHub2024';

app.post('/admin/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    if (email !== ADMIN_EMAIL) return res.status(401).json({ error: 'Invalid admin
credentials' });

    const adminRes = await client.query('SELECT id, password_hash FROM admin_users WHERE
email = $1', [email]);
    if (adminRes.rows.length === 0) return res.status(401).json({ error: 'Admin not found'
});

    const admin = adminRes.rows[0];
```

```
    const validPassword = await bcrypt.compare(password, admin.password_hash);
    if (!validPassword) return res.status(401).json({ error: 'Invalid password' });

    const token = jwt.sign({ adminId: admin.id, email }, JWT_SECRET, { expiresIn: '7d' });
    res.json({ token, message: 'Admin logged in' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

function verifyAdmin(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    if (!decoded.adminId) return res.status(401).json({ error: 'Not an admin token' });
    req.adminId = decoded.adminId;
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

app.post('/admin/reset-services', async (req, res) => {
  try {
    // Add country and city columns if they don't exist
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS country
VARCHAR(100)`);
    await client.query(`ALTER TABLE services ADD COLUMN IF NOT EXISTS city VARCHAR(100)`);
    console.log('? Columns added');

    // Delete all bookings and payments first (cascade)
    await client.query('DELETE FROM payments');
    await client.query('DELETE FROM bookings');
    await client.query('DELETE FROM services');
    console.log('? Old services, bookings, and payments deleted');
    // Insert new country-specific services
    await client.query(`INSERT INTO services (id, name, description, category,
duration_minutes, price, country, city) VALUES
      ('550e8400-e29b-41d4-a716-446655440001', 'Haircut', 'Professional haircut service',
'beauty', 30, 25.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440002', 'Massage Therapy', 'Relaxing full-body
massage', 'wellness', 60, 80.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440003', 'Hair Styling', 'Professional hair
styling', 'beauty', 45, 50.00, 'Italy', 'Florence'),
      ('550e8400-e29b-41d4-a716-446655440100', 'Pizza Delivery', 'Authentic Italian pizza
delivery', 'food', 30, 15.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440101', 'Pasta Making Class', 'Learn traditional
pasta making', 'food', 120, 45.00, 'Italy', 'Bologna'),
      ('550e8400-e29b-41d4-a716-446655440102', 'Wine Tasting Tour', 'Tuscan wine tasting
experience', 'tour', 180, 75.00, 'Italy', 'Florence'),
      ('550e8400-e29b-41d4-a716-446655440103', 'Espresso Masterclass', 'Learn Italian
espresso making', 'food', 60, 35.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440104', 'City Taxi', 'Economy taxi service',
'taxi', 30, 15.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440105', 'Premium Taxi', 'Luxury taxi service',
'taxi', 30, 30.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440106', 'Economy Car Rental - Daily', 'Affordable
economy car rental', 'carrent', 1440, 45.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440107', 'SUV Rental - Daily', 'Large SUV for family
trips', 'carrent', 1440, 85.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440108', '5-Star Hotel', 'Luxury hotel with all
amenities', 'hotel', 1440, 250.00, 'Italy', 'Venice'),
      ('550e8400-e29b-41d4-a716-446655440109', '4-Star Hotel', 'Comfortable hotel with
```

```
great service', 'hotel', 1440, 150.00, 'Italy', 'Florence'),
      ('550e8400-e29b-41d4-a716-446655440110', 'Budget Hotel', 'Affordable hotel rooms',
'hotel', 1440, 75.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440111', 'Gym Membership - Monthly', 'Full month gym
access', 'fitness', 2880, 45.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440112', 'Personal Training - 1 Hour', 'One-on-one
personal training', 'fitness', 60, 70.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440113', 'Yoga Class - 1 Hour', 'Relaxing yoga for
all levels', 'fitness', 60, 30.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440114', 'Pizza Making Class', 'Traditional pizza
making in Naples', 'food', 120, 55.00, 'Italy', 'Naples'),
      ('550e8400-e29b-41d4-a716-446655440115', 'Gondola Tour', 'Classic gondola ride in
Venice', 'tour', 60, 80.00, 'Italy', 'Venice'),
      ('550e8400-e29b-41d4-a716-446655440116', 'Art Class', 'Renaissance art appreciation
class', 'art', 90, 60.00, 'Italy', 'Florence'),
      ('550e8400-e29b-41d4-a716-446655440200', 'Haircut', 'Professional haircut service',
'beauty', 30, 28.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440201', 'Spa Treatment', 'Full body spa treatment',
'wellness', 90, 120.00, 'USA', 'Los Angeles'),
      ('550e8400-e29b-41d4-a716-446655440202', 'Personal Trainer', 'One-on-one fitness
session', 'fitness', 60, 80.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440203', 'Yoga Class', 'Hot yoga session',
'fitness', 60, 25.00, 'USA', 'San Francisco'),
      ('550e8400-e29b-41d4-a716-446655440204', 'Pilates Class', 'Core strengthening
pilates', 'fitness', 60, 35.00, 'USA', 'Los Angeles'),
      ('550e8400-e29b-41d4-a716-446655440205', 'Burger & Fries', 'Gourmet burger
delivery', 'food', 30, 18.00, 'USA', 'Chicago'),
      ('550e8400-e29b-41d4-a716-446655440206', 'Pizza Delivery', 'New York style pizza',
'food', 30, 16.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440207', 'BBQ Catering', 'Authentic BBQ catering
service', 'food', 240, 300.00, 'USA', 'Texas'),
      ('550e8400-e29b-41d4-a716-446655440208', 'Uber/Taxi', 'Ride-sharing service',
'taxi', 20, 12.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440209', 'Car Rental - Daily', 'Daily car rental
service', 'carrent', 1440, 50.00, 'USA', 'Los Angeles'),
      ('550e8400-e29b-41d4-a716-446655440210', 'SUV Rental - Daily', 'Daily SUV rental',
'carrent', 1440, 90.00, 'USA', 'Miami'),
      ('550e8400-e29b-41d4-a716-446655440211', '5-Star Hotel', 'Luxury 5-star hotel',
'hotel', 1440, 280.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440212', '4-Star Hotel', 'Comfortable 4-star hotel',
'hotel', 1440, 180.00, 'USA', 'Los Angeles'),
      ('550e8400-e29b-41d4-a716-446655440213', 'Budget Hotel', 'Affordable budget hotel',
'hotel', 1440, 80.00, 'USA', 'Chicago'),
      ('550e8400-e29b-41d4-a716-446655440214', 'Broadway Show', 'Broadway theater
tickets', 'entertainment', 180, 150.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440215', 'Beach Day Tour', 'Guided beach tour and
activities', 'tour', 480, 120.00, 'USA', 'Miami'),
      ('550e8400-e29b-41d4-a716-446655440216', 'Golden Gate Tour', 'San Francisco city
tour', 'tour', 240, 95.00, 'USA', 'San Francisco'),
      ('550e8400-e29b-41d4-a716-446655440300', 'Web Design', 'Professional website
design', 'freelance', 480, 180.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440301', 'Logo Design', 'Custom logo design
service', 'freelance', 240, 120.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440302', 'Video Editing', 'Professional video
editing', 'freelance', 360, 140.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440303', 'Photography Session', 'Professional photo
shoot', 'freelance', 120, 180.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440304', 'English Tutoring', 'One-on-one English
lessons', 'freelance', 60, 50.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440305', 'Math Tutoring', 'Professional math
tutoring', 'freelance', 60, 55.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440306', 'Piano Lessons', 'Professional piano
instruction', 'freelance', 60, 60.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440307', 'Guitar Lessons', 'Guitar instruction for
all levels', 'freelance', 60, 50.00, 'USA', 'Los Angeles'),
```

```
      ('550e8400-e29b-41d4-a716-446655440308', 'Handyman Service', 'General home repairs',
'freelance', 60, 65.00, 'Italy', 'Rome'),
      ('550e8400-e29b-41d4-a716-446655440309', 'Plumbing Repair', 'Professional plumbing',
'freelance', 120, 100.00, 'USA', 'Chicago'),
      ('550e8400-e29b-41d4-a716-446655440310', 'Electrical Work', 'Licensed electrician
services', 'freelance', 120, 110.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440311', 'House Painting', 'Professional painting
service', 'freelance', 240, 180.00, 'Italy', 'Milan'),
      ('550e8400-e29b-41d4-a716-446655440312', 'Business Consulting', 'Strategic business
advice', 'freelance', 60, 90.00, 'USA', 'San Francisco'),
      ('550e8400-e29b-41d4-a716-446655440313', 'Resume Writing', 'Professional resume
service', 'freelance', 90, 60.00, 'USA', 'New York'),
      ('550e8400-e29b-41d4-a716-446655440400', 'Boulangerie Pastries', 'Fresh French
pastries delivery', 'food', 30, 12.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440401', 'French Cooking Class', 'Learn classic
French cuisine', 'food', 180, 85.00, 'France', 'Lyon'),
      ('550e8400-e29b-41d4-a716-446655440402', 'Wine Tasting - Bordeaux', 'Premium
Bordeaux wine tasting', 'tour', 120, 95.00, 'France', 'Bordeaux'),
      ('550e8400-e29b-41d4-a716-446655440403', 'Eiffel Tower Tour', 'Guided Eiffel Tower
experience', 'tour', 180, 120.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440404', 'Louvre Museum Tour', 'Private museum tour
guide', 'tour', 240, 150.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440405', 'French Massage - 1 Hour', 'Relaxing
therapeutic massage', 'wellness', 60, 90.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440406', 'Luxury Car Rental', 'Premium car rental
service', 'carrent', 1440, 120.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440407', 'Paris Taxi Service', 'Reliable taxi
service', 'taxi', 30, 20.00, 'France', 'Paris'),
      ('550e8400-e29b-41d4-a716-446655440500', 'Bavarian Beer Tour', 'Munich brewery
tour', 'tour', 240, 65.00, 'Germany', 'Munich'),
      ('550e8400-e29b-41d4-a716-446655440501', 'German Schnitzel Class', 'Traditional
schnitzel cooking', 'food', 120, 55.00, 'Germany', 'Berlin'),
      ('550e8400-e29b-41d4-a716-446655440502', 'Bavarian Dance Class', 'Learn traditional
Bavarian dance', 'entertainment', 90, 45.00, 'Germany', 'Munich'),
      ('550e8400-e29b-41d4-a716-446655440503', 'Berlin Wall Tour', 'Historical Berlin Wall
tour', 'tour', 180, 60.00, 'Germany', 'Berlin'),
      ('550e8400-e29b-41d4-a716-446655440504', 'Spa & Sauna Experience', 'Full spa and
sauna day', 'wellness', 180, 120.00, 'Germany', 'Berlin'),
      ('550e8400-e29b-41d4-a716-446655440505', 'Berlin Nightclub Entry', 'VIP nightclub
access', 'entertainment', 480, 85.00, 'Germany', 'Berlin'),
      ('550e8400-e29b-41d4-a716-446655440506', 'Luxury Hotel - 5 Star', 'Premium luxury
accommodation', 'hotel', 1440, 280.00, 'Germany', 'Munich'),
      ('550e8400-e29b-41d4-a716-446655440600', 'Flamenco Lesson', 'Traditional flamenco
dance class', 'entertainment', 90, 50.00, 'Spain', 'Madrid'),
      ('550e8400-e29b-41d4-a716-446655440601', 'Paella Cooking Class', 'Learn authentic
Spanish paella', 'food', 120, 60.00, 'Spain', 'Valencia'),
      ('550e8400-e29b-41d4-a716-446655440602', 'Sagrada Familia Tour', 'Guided Barcelona
cathedral tour', 'tour', 120, 80.00, 'Spain', 'Barcelona'),
      ('550e8400-e29b-41d4-a716-446655440603', 'Park G?ell Tour', 'Guided Gaud? park
experience', 'tour', 180, 95.00, 'Spain', 'Barcelona'),
      ('550e8400-e29b-41d4-a716-446655440604', 'Spanish Guitar Lesson', 'Professional
guitar instruction', 'freelance', 60, 55.00, 'Spain', 'Madrid'),
      ('550e8400-e29b-41d4-a716-446655440605', 'Prado Museum Tour', 'Private art museum
tour', 'tour', 180, 100.00, 'Spain', 'Madrid'),
      ('550e8400-e29b-41d4-a716-446655440606', 'Tapas Food Tour', 'Spanish tapas tasting
tour', 'food', 180, 70.00, 'Spain', 'Barcelona'),
      ('550e8400-e29b-41d4-a716-446655440700', 'London Theatre Show', 'West End theatre
tickets', 'entertainment', 180, 140.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440701', 'Royal Palace Tour', 'Buckingham Palace
guided tour', 'tour', 120, 90.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440702', 'Big Ben & Westminster', 'Parliament
building tour', 'tour', 90, 70.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440703', 'British Tea Experience', 'Afternoon tea
service', 'food', 120, 45.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440704', 'Fish & Chips Cooking', 'Learn traditional
```

```
fish & chips', 'food', 90, 40.00, 'United Kingdom', 'Manchester'),
      ('550e8400-e29b-41d4-a716-446655440705', 'Tower of London Tour', 'Historic tower
tour with crown jewels', 'tour', 180, 85.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440706', 'Scottish Highlands Tour', 'Breathtaking
highlands experience', 'tour', 480, 200.00, 'United Kingdom', 'Edinburgh'),
      ('550e8400-e29b-41d4-a716-446655440707', 'Double-Decker Bus Tour', 'Iconic London
bus tour', 'tour', 120, 35.00, 'United Kingdom', 'London'),
      ('550e8400-e29b-41d4-a716-446655440708', 'Westminster Abbey Tour', 'Historic abbey
guided tour', 'tour', 90, 55.00, 'United Kingdom', 'London')
      ON CONFLICT DO NOTHING`);

    res.json({ message: 'Services reset successfully!', count: 50 });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});;

app.get('/cities', async (req, res) => {
  try {
    const result = await client.query(`
      SELECT country, name FROM cities
      UNION
      SELECT DISTINCT country, city AS name FROM services
      WHERE country IS NOT NULL AND city IS NOT NULL
      ORDER BY country, name
    `);
    res.json(result.rows);
  } catch (err) {
    console.error('Cities error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/provider-integrations/status', (req, res) => {
  res.json({ hotels: { enabled: providerConfig.enabled, provider: 'hotel-adapter' } });
});

app.get('/provider-integrations/hotels/search', async (req, res) => {
  const { destination, checkin, checkout, guests } = req.query;
  if (!destination || !checkin || !checkout) return res.status(400).json({ error:
'destination, checkin, and checkout are required' });
  try {
    res.json(await searchHotels({ destination, checkin, checkout, guests }));
  } catch (err) {
    res.status(502).json({ error: err.message });
  }
});

app.post('/provider-integrations/hotels/reservations', verifyToken, async (req, res) => {
  try {
    res.json(await createHotelReservation({ ...req.body, customer_id: req.userId }));
  } catch (err) {
    res.status(502).json({ error: err.message });
  }
});


app.post('/admin/recurring-schedules', verifyAdmin, async (req, res) => {
  const { service_id, provider_id, weekday, start_time, end_time, slot_minutes = 60,
buffer_minutes = 15, timezone = 'UTC', valid_from, valid_until, capacity = 1, breaks = []
} = req.body;
  if (!service_id || weekday === undefined || !start_time || !end_time || !valid_from)
return res.status(400).json({ error: 'service_id, weekday, start_time, end_time, and
valid_from are required' });
  try {
```

```
    await client.query('BEGIN');
    const schedule = await client.query(`INSERT INTO recurring_schedules (id, service_id,
provider_id, weekday, start_time, end_time, slot_minutes, buffer_minutes, timezone,
valid_from, valid_until, capacity) VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12)
RETURNING *`, [uuidv4(), service_id, provider_id || null, weekday, start_time, end_time,
slot_minutes, buffer_minutes, timezone, valid_from, valid_until || null, capacity]);
    for (const item of breaks) await client.query('INSERT INTO schedule_breaks (id,
schedule_id, break_start, break_end) VALUES ($1,$2,$3,$4)', [uuidv4(),
schedule.rows[0].id, item.start, item.end]);
    await client.query('COMMIT');
    res.status(201).json(schedule.rows[0]);
  } catch (err) { await client.query('ROLLBACK').catch(() => null); res.status(500).json({
error: err.message }); }
});

app.post('/admin/schedule-exceptions', verifyAdmin, async (req, res) => {
  const { service_id, exception_date, closed = true, start_time, end_time } = req.body;
  try {
    const result = await client.query('INSERT INTO schedule_exceptions (id, service_id,
exception_date, closed, start_time, end_time) VALUES ($1,$2,$3,$4,$5,$6) RETURNING *',
[uuidv4(), service_id, exception_date, closed, start_time || null, end_time || null]);
    res.status(201).json(result.rows[0]);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.post('/admin/availability-slots/generate', verifyAdmin, async (req, res) => {
  const { service_id, from, to } = req.body;
  if (!service_id || !from || !to) return res.status(400).json({ error: 'service_id, from,
and to are required' });
  try {
    const schedules = await client.query('SELECT * FROM recurring_schedules WHERE
service_id = $1 AND active = true', [service_id]);
    const breaks = await client.query('SELECT b.* FROM schedule_breaks b JOIN
recurring_schedules s ON s.id = b.schedule_id WHERE s.service_id = $1', [service_id]);
    const exceptions = await client.query('SELECT * FROM schedule_exceptions WHERE
service_id = $1 AND exception_date BETWEEN $2 AND $3', [service_id, from, to]);
    let created = 0; const start = new Date(`${from}T00:00:00Z`); const end = new
Date(`${to}T00:00:00Z`);
    for (let day = new Date(start); day <= end; day.setUTCDate(day.getUTCDate() + 1)) {
      const date = day.toISOString().slice(0, 10); const weekday = day.getUTCDay();
      const exception = exceptions.rows.find(x => String(x.exception_date).slice(0, 10)
=== date);
      if (exception?.closed) continue;
      for (const schedule of schedules.rows.filter(x => x.weekday === weekday && date >=
String(x.valid_from).slice(0, 10) && (!x.valid_until || date <=
String(x.valid_until).slice(0, 10)))) {
        const [sh, sm] = String(schedule.start_time).split(':').map(Number); const [eh,
em] = String(schedule.end_time).split(':').map(Number); const [slot, step] = [new
Date(Date.UTC(1970,0,1,sh,sm)), (Number(schedule.slot_minutes) +
Number(schedule.buffer_minutes)) * 60000];
        const finish = new Date(Date.UTC(1970,0,1,eh,em));
        for (; slot < finish; slot.setTime(slot.getTime() + step)) { const endSlot = new
Date(slot.getTime() + Number(schedule.slot_minutes) * 60000); if (endSlot > finish) break;
const st = slot.toISOString().slice(11,19); const et = endSlot.toISOString().slice(11,19);
const blocked = breaks.rows.some(b => b.schedule_id === schedule.id && st < b.break_end &&
et > b.break_start); if (blocked) continue; const exists = await client.query('SELECT 1
FROM availability_slots WHERE service_id=$1 AND slot_date=$2 AND start_time=$3',
[service_id, date, st]); if (!exists.rows.length) { await client.query('INSERT INTO
availability_slots
(id,service_id,provider_id,slot_date,start_time,end_time,timezone,capacity) VALUES
($1,$2,$3,$4,$5,$6,$7,$8)', [uuidv4(), service_id, schedule.provider_id, date, st, et,
schedule.timezone, schedule.capacity]); created++; } }
      }
    }
    res.json({ created, from, to });
```

```
  } catch (err) { res.status(500).json({ error: err.message }); }
});
app.get('/availability/:service_id', async (req, res) => {
  const { from, to } = req.query;
  try {
    const result = await client.query(
      `SELECT s.id, s.slot_date, s.start_time, s.end_time, s.timezone, s.capacity,
              s.capacity - COUNT(b.id) FILTER (WHERE b.status IN ('pending','confirmed'))
AS remaining
       FROM availability_slots s LEFT JOIN bookings b ON b.service_id = s.service_id
       AND b.booking_date = s.slot_date AND b.booking_time >= s.start_time AND
b.booking_time < s.end_time
       WHERE s.service_id = $1 AND s.active = true
       AND ($2::date IS NULL OR s.slot_date >= $2::date) AND ($3::date IS NULL OR
s.slot_date <= $3::date)
       GROUP BY s.id ORDER BY s.slot_date, s.start_time`,
      [req.params.service_id, from || null, to || null]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.post('/admin/availability-slots', verifyAdmin, async (req, res) => {
  const { service_id, provider_id, slot_date, start_time, end_time, timezone = 'UTC',
capacity = 1 } = req.body;
  if (!service_id || !slot_date || !start_time || !end_time) return res.status(400).json({
error: 'service_id, slot_date, start_time, and end_time are required' });
  try {
    const result = await client.query(
      `INSERT INTO availability_slots (id, service_id, provider_id, slot_date, start_time,
end_time, timezone, capacity)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING *`,
      [uuidv4(), service_id, provider_id || null, slot_date, start_time, end_time,
timezone, Math.max(1, Number(capacity))]
    );
    res.status(201).json(result.rows[0]);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.delete('/admin/availability-slots/:id', verifyAdmin, async (req, res) => {
  try {
    await client.query('UPDATE availability_slots SET active = false WHERE id = $1',
[req.params.id]);
    res.json({ message: 'Availability slot disabled' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.get('/services', async (req, res) => {
  try {
    const result = await client.query(`
      SELECT id, name, description, price, country, city, availability_status,
             COALESCE((SELECT verification_status FROM users WHERE users.id =
services.provider_id), 'unverified') AS provider_verification_status
      FROM services
      ORDER BY country, city, name
    `);
    res.json(result.rows);
  } catch (err) {
    console.error('Services error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/services-by-location', async (req, res) => {
  try {
```

```
    const { country, city } = req.query;
    const params = [];
    const filters = [];

    if (country) {
      params.push(country);
      filters.push(`country = $${params.length}`);
    }

    if (city) {
      params.push(city);
      filters.push(`city = $${params.length}`);
    }

    const where = filters.length ? `WHERE ${filters.join(' AND ')}` : '';

    const result = await client.query(`
      SELECT s.id, s.name, s.description, s.price, s.country, s.city,
s.availability_status,
             s.listing_type, s.rental_unit, s.approval_status,
             COALESCE(u.verification_status, 'unverified') AS
provider_verification_status,
             u.id AS provider_id, u.name AS provider_name,
             COALESCE(s.available_count, 0) AS available_count,
             COALESCE((SELECT COUNT(*) FROM availability_slots a WHERE a.service_id=s.id
AND a.active=true AND a.slot_date >= CURRENT_DATE), 0) AS live_slot_count
      FROM services s LEFT JOIN users u ON u.id=s.provider_id
      ${where}
      ORDER BY s.name
    `, params);
    res.json(result.rows);
  } catch (err) {
    console.error('Location services error:', err);
    res.status(500).json({ error: err.message });
  }
});

app.get('/admin/dashboard', verifyAdmin, async (req, res) => {
  try {
    const totalUsers = await client.query('SELECT COUNT(*) FROM users');
    const totalBookings = await client.query('SELECT COUNT(*) FROM bookings');
    const totalPayments = await client.query('SELECT SUM(total_paid) as total FROM
payments');
    const totalEarnings = await client.query('SELECT total_commission FROM earnings');

    res.json({
      totalUsers: parseInt(totalUsers.rows[0].count),
      totalBookings: parseInt(totalBookings.rows[0].count),
      totalPayments: parseFloat(totalPayments.rows[0]?.total || 0),
      totalEarnings: parseFloat(totalEarnings.rows[0]?.total_commission || 0)
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/admin/users', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query('SELECT id, email, name, created_at FROM users ORDER
BY created_at DESC LIMIT 100');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.get('/admin/bookings', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query(`
      SELECT b.id, b.user_id, b.booking_date, b.status, b.address, b.city,
        s.name as service_name, s.price,
        u.name as user_name, u.email
      FROM bookings b
      JOIN services s ON b.service_id = s.id
      JOIN users u ON b.user_id = u.id
      ORDER BY b.created_at DESC
      LIMIT 100
    `);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/admin/payments', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query(`
      SELECT p.id, p.service_price, p.commission, p.total_paid, p.payment_status,
p.created_at,
        b.city, s.name as service_name,
        u.name as user_name, u.email
      FROM payments p
      JOIN bookings b ON p.booking_id = b.id
      JOIN services s ON b.service_id = s.id
      JOIN users u ON p.user_id = u.id
      ORDER BY p.created_at DESC
      LIMIT 100
    `);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/admin/subscriptions', verifyAdmin, async (req, res) => {
  try {
    const result = await client.query(`
      SELECT s.id, s.user_id, s.plan, s.booking_limit, s.monthly_bookings,
s.renewal_price, s.auto_renew, s.subscription_date,
        u.name, u.email
      FROM subscriptions s
      JOIN users u ON s.user_id = u.id
      ORDER BY s.subscription_date DESC
      LIMIT 100
    `);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.patch('/admin/bookings/:id/status', verifyAdmin, async (req, res) => {
  const { id } = req.params;
  const { status } = req.body;

  try {
    if (!['pending', 'confirmed', 'cancelled'].includes(status)) {
      return res.status(400).json({ error: 'Invalid status' });
    }
```

```
    await client.query('UPDATE bookings SET status = $1 WHERE id = $2', [status, id]);
    res.json({ message: 'Booking status updated' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/', (req, res) => res.sendFile(__dirname + '/index.html'));
app.get('/admin', (req, res) => res.sendFile(__dirname + '/admin.html'));

const PORT = process.env.PORT || 3001;
// Subscription endpoints
const SUBSCRIPTION_PLANS = {
  free: { name: 'Free', booking_limit: 10, renewal_price: 0 },
  premium: { name: 'Premium', booking_limit: 50, renewal_price: 9.99 },
  business: { name: 'Business', booking_limit: 200, renewal_price: 29.99 }
};

app.get('/subscriptions/plans', (req, res) => {
  res.json(Object.entries(SUBSCRIPTION_PLANS).map(([id, plan]) => ({ id, ...plan })));
});

app.get('/subscriptions/me', verifyToken, async (req, res) => {
  try {
    const result = await client.query(
      `SELECT id, plan, booking_limit, monthly_bookings, subscription_date, next_reset,
renewal_price, auto_renew
       FROM subscriptions WHERE user_id = $1`,
      [req.userId]
    );
    if (result.rows.length === 0) {
      const created = await client.query(
        `INSERT INTO subscriptions (id, user_id, plan, booking_limit, renewal_price)
         VALUES ($1, $2, 'free', $3, $4)
         RETURNING id, plan, booking_limit, monthly_bookings, subscription_date,
next_reset, renewal_price, auto_renew`,
        [uuidv4(), req.userId, SUBSCRIPTION_PLANS.free.booking_limit,
SUBSCRIPTION_PLANS.free.renewal_price]
      );
      return res.json(created.rows[0]);
    }
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/subscriptions/upgrade', verifyToken, async (req, res) => {
  const { plan } = req.body;
  const selected = SUBSCRIPTION_PLANS[plan];
  if (!selected || plan === 'free') return res.status(400).json({ error: 'Choose premium
or business' });
  try {
    const result = await client.query(
      `INSERT INTO subscriptions (id, user_id, plan, booking_limit, renewal_price)
       VALUES ($1, $2, $3, $4, $5)
       ON CONFLICT (user_id) DO UPDATE SET plan = EXCLUDED.plan, booking_limit =
EXCLUDED.booking_limit,
       renewal_price = EXCLUDED.renewal_price, subscription_date = CURRENT_TIMESTAMP,
next_reset = CURRENT_TIMESTAMP + INTERVAL '1 month'
       RETURNING id, plan, booking_limit, monthly_bookings, subscription_date, next_reset,
renewal_price, auto_renew`,
      [uuidv4(), req.userId, plan, selected.booking_limit, selected.renewal_price]
    );
    res.json(result.rows[0]);
```

```
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.listen(PORT, () => {
  console.log(`Backend running on port ${PORT}`);
  initDB();
});

client.on('error', (err) => console.error('Database error:', err));


app.post('/withdrawals/request', async (req, res) => {
  const { amount, paypal_email } = req.body;
  if (!amount || amount <= 0) return res.status(400).json({ error: 'Invalid amount' });
  try {
    const withdrawalId = uuidv4();
    await sendEmail(process.env.EMAIL_USER, 'New Withdrawal Request', `<h2>Withdrawal
Request</h2><p>Amount: $${amount}</p><p>PayPal: ${paypal_email}</p>`).catch(() => null);
    res.json({ message: 'Withdrawal request submitted. Email notification sent.',
withdrawalId, amount, status: 'pending' });
  } catch (err) {
    res.json({ message: 'Request submitted', amount });
  }
});

app.post('/withdrawals/:id/process', async (req, res) => {
  const { id } = req.params;
  const { transactionId } = req.body;
  try {
    const txId = transactionId || 'TX_' + Date.now();
    const approvalEmail = `<h2>Withdrawal Approved! ?</h2><p>Your withdrawal has been
processed.</p><p><strong>Transaction ID:</strong> ${txId}</p><p>The funds will arrive in
your PayPal account within 24-48 hours.</p>`;
    await sendEmail(process.env.EMAIL_USER, 'Withdrawal Approved', approvalEmail).catch(()
=> null);
    res.json({ message: 'Withdrawal approved! Email sent to admin.', transactionId: txId,
status: 'approved' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/withdrawals/:id/reject', async (req, res) => {
  const { id } = req.params;
  const { notes } = req.body;
  try {
    const rejectEmail = `<h2>Withdrawal Rejected ?</h2><p>Your withdrawal request has been
rejected.</p><p><strong>Reason:</strong> ${notes || 'Not specified'}</p><p>Please contact
support for more information.</p>`;
    await sendEmail(process.env.EMAIL_USER, 'Withdrawal Rejected', rejectEmail).catch(()
=> null);
    res.json({ message: 'Withdrawal rejected. Email notification sent.', status:
'rejected' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/withdrawals/history', async (req, res) => {
  res.json([{
    id: '1',
    amount: 500,
    status: 'pending',
```

```
    request_date: new Date(),
    paypal_email: process.env.ADMIN_PAYPAL_EMAIL || 'admin@bookhub.app',
    transaction_id: null
  }]);
});


app.get('/admin/reseed-bologna', async (req, res) => { try { await client.query('DELETE
FROM services WHERE city = ', ['Bologna']); const services = [{id: '550e8400-e29b-41d4-
a716-446655440500', name: 'Tagliatelle al Rag? Cooking Class', desc: 'Learn traditional
Bolognese pasta', cat: 'food', dur: 120, price: 85}, {id: '550e8400-e29b-41d4-a716-
446655440501', name: 'Tortellini Making', desc: 'Handmade tortellini', cat: 'food', dur:
120, price: 80}, {id: '550e8400-e29b-41d4-a716-446655440502', name: 'Mortadella Tasting',
desc: 'Authentic mortadella', cat: 'food', dur: 90, price: 65}, {id: '550e8400-e29b-41d4-
a716-446655440503', name: 'Balsamic Vinegar Tour', desc: 'Vinegar makers', cat: 'tour',
dur: 180, price: 95}, {id: '550e8400-e29b-41d4-a716-446655440504', name: 'Bologna Walking
Tour', desc: 'Historic Bologna', cat: 'tour', dur: 180, price: 75}, {id: '550e8400-e29b-
41d4-a716-446655440505', name: 'Parmesan Factory', desc: 'Cheese production', cat: 'tour',
dur: 240, price: 110}, {id: '550e8400-e29b-41d4-a716-446655440506', name: 'Wine Tasting',
desc: 'Emilia wines', cat: 'beverage', dur: 120, price: 70}, {id: '550e8400-e29b-41d4-
a716-446655440507', name: 'Pasta Art', desc: 'Pasta decoration', cat: 'art', dur: 90,
price: 60}, {id: '550e8400-e29b-41d4-a716-446655440508', name: 'Pizza Oven', desc: 'Wood-
fired pizza', cat: 'food', dur: 120, price: 75}, {id: '550e8400-e29b-41d4-a716-
446655440509', name: 'Gelato Making', desc: 'Gelato & chocolate', cat: 'food', dur: 90,
price: 70}]; for (const s of services) { await client.query('INSERT INTO services (id,
name, description, category, duration_minutes, price, country, city) VALUES ($1, $2, $3,
$4, $5, $6, $7, $8)', [s.id, s.name, s.desc, s.cat, s.dur, s.price, 'Italy', 'Bologna']);
} res.json({ message: '10 Bologna services added!' }); } catch (err) {
res.status(500).json({ error: err.message }); } });



app.get('/admin/reseed-all-services', async (req, res) => {
  try {
    await client.query('DELETE FROM services');
    const realServices = [
      {id: '1', name: 'Professional Haircut', desc: 'Premium haircut with styling', cat:
'beauty', dur: 45, price: 35, country: 'USA', city: 'New York'},
      {id: '2', name: 'Full Body Massage', desc: '60min relaxing massage', cat:
'wellness', dur: 60, price: 90, country: 'USA', city: 'Los Angeles'},
      {id: '3', name: 'Yoga Class', desc: 'Beginner yoga session', cat: 'wellness', dur:
60, price: 25, country: 'USA', city: 'Austin'},
      {id: '4', name: 'Personal Training', desc: '1-on-1 fitness session', cat: 'fitness',
dur: 60, price: 60, country: 'USA', city: 'Miami'},
      {id: '5', name: 'Home Cleaning', desc: 'Deep house cleaning', cat: 'home', dur: 180,
price: 120, country: 'USA', city: 'Chicago'},
      {id: '6', name: 'Plumbing Repair', desc: 'Emergency plumbing fix', cat: 'repair',
dur: 120, price: 85, country: 'USA', city: 'Houston'},
      {id: '7', name: 'Web Design', desc: 'Custom website design', cat: 'tech', dur: 480,
price: 250, country: 'USA', city: 'San Francisco'},
      {id: '8', name: 'Photography Session', desc: '2-hour photo shoot', cat: 'media',
dur: 120, price: 150, country: 'USA', city: 'Seattle'},
      {id: '9', name: 'Piano Lessons', desc: 'Professional piano instruction', cat:
'music', dur: 60, price: 50, country: 'USA', city: 'Boston'},
      {id: '10', name: 'Language Tutoring', desc: 'English language lessons', cat:
'education', dur: 60, price: 40, country: 'USA', city: 'Denver'},
      {id: '11', name: 'Italian Cooking Class', desc: 'Learn authentic Italian cuisine',
cat: 'food', dur: 120, price: 75, country: 'Italy', city: 'Rome'},
      {id: '12', name: 'Pasta Making', desc: 'Handmade fresh pasta', cat: 'food', dur: 90,
price: 65, country: 'Italy', city: 'Bologna'},
      {id: '13', name: 'Wine Tasting Tour', desc: 'Tuscan wine experience', cat: 'tour',
dur: 180, price: 95, country: 'Italy', city: 'Florence'},
      {id: '14', name: 'Gelato Making', desc: 'Learn to make Italian gelato', cat: 'food',
dur: 90, price: 70, country: 'Italy', city: 'Milan'},
      {id: '15', name: 'Art Class', desc: 'Classical painting techniques', cat: 'art',
```

```
dur: 120, price: 55, country: 'Italy', city: 'Venice'},
      {id: '16', name: 'French Cooking', desc: 'French culinary techniques', cat: 'food',
dur: 150, price: 120, country: 'France', city: 'Paris'},
      {id: '17', name: 'Wine Sommelier', desc: 'Wine pairing session', cat: 'beverage',
dur: 90, price: 110, country: 'France', city: 'Bordeaux'},
      {id: '18', name: 'Seine River Tour', desc: 'Guided Paris boat tour', cat: 'tour',
dur: 120, price: 45, country: 'France', city: 'Paris'},
      {id: '19', name: 'Flamenco Dancing', desc: 'Spanish dance lessons', cat: 'dance',
dur: 90, price: 60, country: 'Spain', city: 'Madrid'},
      {id: '20', name: 'Spanish Guitar', desc: 'Guitar instruction', cat: 'music', dur:
60, price: 55, country: 'Spain', city: 'Barcelona'},
      {id: '21', name: 'Sushi Making', desc: 'Japanese sushi preparation', cat: 'food',
dur: 120, price: 85, country: 'Japan', city: 'Tokyo'},
      {id: '22', name: 'Tea Ceremony', desc: 'Traditional Japanese tea', cat: 'culture',
dur: 90, price: 50, country: 'Japan', city: 'Kyoto'},
      {id: '23', name: 'Martial Arts', desc: 'Karate training', cat: 'fitness', dur: 60,
price: 45, country: 'Japan', city: 'Tokyo'},
      {id: '24', name: 'Surfing Lessons', desc: 'Beach surfing instruction', cat:
'sports', dur: 120, price: 80, country: 'Australia', city: 'Sydney'},
      {id: '25', name: 'Scuba Diving', desc: 'Guided scuba experience', cat: 'adventure',
dur: 180, price: 150, country: 'Australia', city: 'Great Barrier Reef'},
      {id: '26', name: 'Yoga Retreat', desc: '3-day wellness retreat', cat: 'wellness',
dur: 2880, price: 400, country: 'Thailand', city: 'Bangkok'},
      {id: '27', name: 'Thai Massage', desc: 'Traditional Thai massage', cat: 'wellness',
dur: 90, price: 35, country: 'Thailand', city: 'Bangkok'},
      {id: '28', name: 'Cooking Class', desc: 'Thai cooking techniques', cat: 'food', dur:
120, price: 45, country: 'Thailand', city: 'Chiang Mai'},
      {id: '29', name: 'Salsa Dancing', desc: 'Latin dance lessons', cat: 'dance', dur:
60, price: 40, country: 'Brazil', city: 'Rio de Janeiro'},
      {id: '30', name: 'Capoeira Training', desc: 'Brazilian martial art', cat: 'fitness',
dur: 60, price: 50, country: 'Brazil', city: 'Salvador'}
    ];
    for (const s of realServices) {
      const uuid = require('uuid').v4();
      await client.query('INSERT INTO services (id, name, description, category,
duration_minutes, price, country, city) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)', [uuid,
s.name, s.desc, s.cat, s.dur, s.price, s.country, s.city]).catch(() => null);
    }
    res.json({ message: '30 real services added across 10 countries!' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});


app.get('/admin/load-real-services', async (req, res) => {
  try {
    // Delete in correct order to avoid FK constraints
    await client.query('DELETE FROM payments');
    await client.query('DELETE FROM bookings');
    await client.query('DELETE FROM services');

    // Insert new real services
    const services = [
      ['10001', 'Home Cleaning - 2 Hours', 'Professional house cleaning service',
'cleaning', 120, 45, 'USA', 'New York'],
      ['10002', 'Deep Cleaning - 3 Hours', 'Deep clean entire apartment', 'cleaning', 180,
65, 'USA', 'New York'],
      ['10003', 'Move-In/Out Cleaning', 'Complete apartment cleaning after move',
'cleaning', 240, 85, 'USA', 'Los Angeles'],
      ['10004', 'Kitchen Deep Clean', 'Professional kitchen cleaning', 'cleaning', 90, 40,
'USA', 'Chicago'],
      ['10005', 'Bathroom Cleaning', 'Complete bathroom sanitation', 'cleaning', 60, 35,
'USA', 'Houston'],
      ['20001', 'Plumbing Repair', 'Fix leaks, unclog drains', 'repairs', 120, 75, 'USA',
```

```
'New York'],
      ['20002', 'Electrical Repair', 'Fix outlets, switches, wiring', 'repairs', 120, 85,
'USA', 'Los Angeles'],
      ['20003', 'Door/Lock Installation', 'Install new doors or locks', 'repairs', 90, 55,
'USA', 'Chicago'],
      ['20004', 'Drywall Repair', 'Fix holes and patch drywall', 'repairs', 120, 65,
'USA', 'Houston'],
      ['20005', 'Appliance Repair', 'Fix washer, dryer, dishwasher', 'repairs', 120, 95,
'USA', 'Phoenix'],
      ['30001', 'Moving Help - 2 Hours', 'Help loading/unloading truck', 'moving', 120,
50, 'USA', 'New York'],
      ['30002', 'Furniture Delivery', 'Deliver and assemble furniture', 'moving', 120, 60,
'USA', 'Los Angeles'],
      ['30003', 'Junk Removal', 'Remove and haul away junk', 'moving', 180, 80, 'USA',
'Chicago'],
      ['30004', 'Piano Moving', 'Specialized piano moving service', 'moving', 240, 150,
'USA', 'Houston'],
      ['40001', 'Haircut & Styling', 'Professional haircut', 'beauty', 45, 30, 'USA', 'New
York'],
      ['40002', 'Full Body Massage', 'Relaxing therapeutic massage', 'wellness', 60, 80,
'USA', 'Los Angeles'],
      ['40003', 'Manicure & Pedicure', 'Professional nail care', 'beauty', 90, 45, 'USA',
'Chicago'],
      ['40004', 'Haircut for Kids', 'Kids haircut service', 'beauty', 30, 20, 'USA',
'Houston'],
      ['40005', 'Waxing Service', 'Hair removal waxing', 'beauty', 45, 35, 'USA',
'Phoenix'],
      ['50001', 'Dog Walking - 30 min', 'Walk your dog', 'pets', 30, 20, 'USA', 'New
York'],
      ['50002', 'Dog Grooming', 'Full dog grooming service', 'pets', 120, 60, 'USA', 'Los
Angeles'],
      ['50003', 'Pet Sitting - 1 Hour', 'Watch your pet while away', 'pets', 60, 25,
'USA', 'Chicago'],
      ['50004', 'Dog Training Session', 'One-on-one dog training', 'pets', 60, 55, 'USA',
'Houston'],
      ['50005', 'Pet Boarding - Per Night', 'Overnight pet care', 'pets', 1440, 45, 'USA',
'Phoenix'],
      ['60001', 'Math Tutoring - 1 Hour', 'One-on-one math lessons', 'education', 60, 40,
'USA', 'New York'],
      ['60002', 'English Tutoring - 1 Hour', 'ESL or writing lessons', 'education', 60,
38, 'USA', 'Los Angeles'],
      ['60003', 'Programming Lesson', 'Learn coding basics', 'education', 60, 55, 'USA',
'Chicago'],
      ['60004', 'SAT Prep - 2 Hours', 'College entrance exam prep', 'education', 120, 70,
'USA', 'Houston'],
      ['60005', 'Music Lessons - Piano', 'One-on-one piano instruction', 'education', 60,
50, 'USA', 'Phoenix'],
      ['70001', 'Personal Training Session', '1-on-1 fitness training', 'fitness', 60, 65,
'USA', 'New York'],
      ['70002', 'Yoga Class', 'Group or private yoga', 'fitness', 60, 30, 'USA', 'Los
Angeles'],
      ['70003', 'HIIT Training', 'High-intensity interval training', 'fitness', 45, 50,
'USA', 'Chicago'],
      ['70004', 'Pilates Class', 'Core strengthening class', 'fitness', 60, 35, 'USA',
'Houston'],
      ['70005', 'Boxing Training', 'One-on-one boxing lessons', 'fitness', 60, 60, 'USA',
'Phoenix'],
      ['80001', 'Professional Photo Shoot - 1 Hour', 'Portrait or event photography',
'photography', 60, 120, 'USA', 'New York'],
      ['80002', 'Photo Editing Service', 'Edit and enhance your photos', 'photography',
120, 50, 'USA', 'Los Angeles'],
      ['80003', 'Headshot Photography', 'Professional headshots', 'photography', 60, 100,
'USA', 'Chicago'],
      ['80004', 'Family Photo Session - 2 Hours', 'Family portrait session',
'photography', 120, 150, 'USA', 'Houston'],
```

```
      ['90001', 'Computer Repair', 'Fix slow/broken computers', 'tech', 120, 70, 'USA',
'New York'],
      ['90002', 'Website Design', 'Custom website creation', 'tech', 480, 250, 'USA', 'Los
Angeles'],
      ['90003', 'Social Media Management - Monthly', 'Manage business social media',
'tech', 2880, 200, 'USA', 'Chicago'],
      ['90004', 'Logo Design', 'Custom business logo', 'tech', 240, 100, 'USA',
'Houston'],
      ['90005', 'Video Editing - Per Hour', 'Professional video editing', 'tech', 60, 80,
'USA', 'Phoenix'],
      ['it001', 'Home Cleaning - Roma', 'Professional cleaning in Rome', 'cleaning', 120,
35, 'Italy', 'Rome'],
      ['it002', 'Cooking Class - Pasta Making', 'Learn traditional Italian pasta', 'food',
120, 70, 'Italy', 'Bologna'],
      ['it003', 'Wine Tasting Tour', 'Tuscan wine experience', 'tour', 180, 90, 'Italy',
'Florence'],
      ['it004', 'Plumbing Service - Milano', 'Emergency plumbing repair', 'repairs', 120,
65, 'Italy', 'Milan'],
      ['it005', 'Massage Therapy - Venezia', 'Relaxing massage in Venice', 'wellness', 60,
70, 'Italy', 'Venice'],
      ['fr001', 'French Cooking Class', 'Learn French cuisine', 'food', 150, 100,
'France', 'Paris'],
      ['fr002', 'Seine River Tour', 'Guided Paris boat tour', 'tour', 120, 50, 'France',
'Paris'],
      ['fr003', 'Wine Sommelier Session', 'Wine pairing experience', 'food', 90, 110,
'France', 'Bordeaux'],
      ['fr004', 'Home Cleaning - Paris', 'Professional apartment cleaning', 'cleaning',
120, 50, 'France', 'Paris'],
      ['es001', 'Flamenco Dancing Lessons', 'Traditional Spanish dance', 'dance', 90, 60,
'Spain', 'Madrid'],
      ['es002', 'Spanish Guitar Lessons', 'Guitar instruction', 'music', 60, 50, 'Spain',
'Barcelona'],
      ['es003', 'Paella Cooking Class', 'Learn traditional paella', 'food', 120, 75,
'Spain', 'Valencia'],
      ['es004', 'Home Cleaning - Barcelona', 'Professional cleaning', 'cleaning', 120, 40,
'Spain', 'Barcelona'],
      ['jp001', 'Sushi Making Class', 'Learn authentic sushi prep', 'food', 120, 85,
'Japan', 'Tokyo'],
      ['jp002', 'Tea Ceremony Experience', 'Traditional Japanese tea', 'culture', 90, 60,
'Japan', 'Kyoto'],
      ['jp003', 'Karate Training', 'Martial arts instruction', 'fitness', 60, 50, 'Japan',
'Tokyo'],
      ['jp004', 'Home Cleaning - Tokyo', 'Professional apartment cleaning', 'cleaning',
120, 45, 'Japan', 'Tokyo']
    ];

    for (const s of services) {
      const uuid = uuidv4();
      await client.query('INSERT INTO services (id, name, description, category,
duration_minutes, price, country, city) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)', [uuid,
s[1], s[2], s[3], s[4], s[5], s[6], s[7]]).catch(e => null);
    }

    res.json({ message: 'Successfully loaded 60 REAL marketplace services across 15
cities!', count: services.length });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
client.on('error', (err) => console.error('Database error:', err));
```